Sustavi baza podataka

Predavanja

1. Arhitektura sustava

ožujak 2026.

Sustavi baza podataka

Baza podataka

- BAZA PODATAKA je skup podataka koji su pohranjeni i organizirani tako da mogu zadovoljiti zahtjeve korisnika.

  (*M. Vetter, 1981.*)

- BAZA PODATAKA je skup perzistentnih podataka kojeg koriste aplikacijski sustavi neke organizacije.

  (*C. J. Date, 2000.*)

- BAZA PODATAKA je skup međusobno povezanih podataka, pohranjenih zajedno, uz isključenje bespotrebne zalihosti (redundancije), koji mogu zadovoljiti različite primjene. Podaci su pohranjeni na način neovisan o programima koji ih koriste. Prilikom dodavanja novih podataka, mijenjanja i pretraživanja postojećih podataka primjenjuje se zajednički i kontrolirani pristup. Podaci su strukturirani tako da služe kao osnova za razvoj budućih primjena.

  (*J. Martin, 1979.*)

Sustav za upravljanje bazama podataka

- programski sustav koji upravlja istovremenim pristupom bazi podataka od strane više korisnika/aplikacija uz osiguravanje sigurnosti i integriteta baze podataka
  - složeni programski sustav - obuhvaća konglomerat znanja i desetljećima razvijanih koncepata i tehnologija
  - jedan od najranije razvijenih višekorisničkih sustava
    - utjecaj na aplikacijsku programsku potporu, operacijske sustave, mrežne servise
- relacijski sustav za upravljanje bazama podataka

Zadaće sustava za upravljanje bazama podataka

- trajna pohrana podataka (*persistent storage*)
- osiguravanje programskog sučelja (*programming interface*)
  - DDL (Data Definition Language), DML (Data Manipulation Language)
- zaštita podataka
  - integritet podataka (*integrity*)
  - pristup podacima - autorizacija, sigurnost (*security*)
  - potpora za upravljanje transakcijama
    - upravljanje istodobnim pristupom (*concurrency control*)
    - obnova u slučaju razrušenja (*recovery*)
- optimiranje metoda pristupa podacima (*query optimization*)

Sustav baza podataka (Database system)

- Sustav za upravljanje bazama podataka zajedno s bazom podataka (i aplikacijama) čini sustav baze podataka

  [Elmasri, 2000.]

- Sustav baze podataka je kolokvijalni naziv za sustav za upravljanje bazama podataka

  [Garcia-Molina, 2000.]

Glavne zadaće administratora baze podataka

Informativno

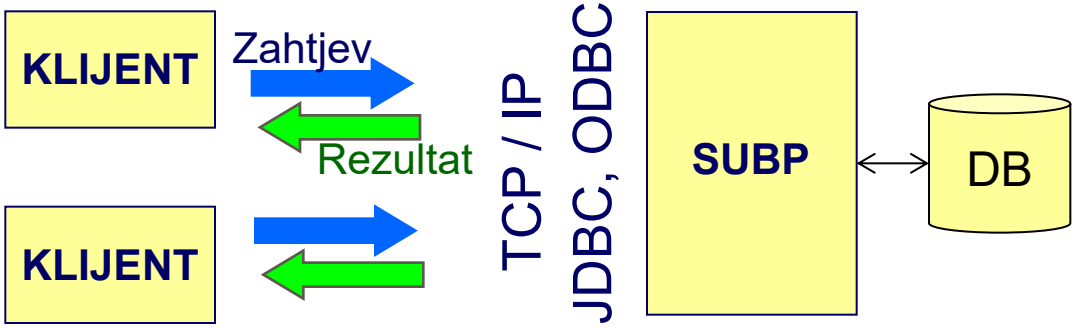
- instalacija i održavanje programske potpore koja se odnosi na sustav za upravljanje bazama podataka (*database engine* + *utilities*)
- održavanje sheme baze podataka
- nadzor pristupa podacima (autorizacija)
- planiranje i održavanje prostora za pohranu
  - sekundarna i tercijarna memorija
- nadzor performansi sustava
  - optimizacija upita
  - podešavanje parametara sustava (veličina međuspremnika, veličina i broj logičkih dnevnika, veličina tablice ključeva, alokacija procesora, metoda detekcije potpunih zastoja, ...)
- priprema procedura za obnovu podataka (i obnova podataka po potrebi)
  - upravljanje arhivskim kopijama i logičkim dnevnicima
- upravljanje i nadzor nad procesima replikacije i distribucije podataka
- komunikacija s korisnicima (aplikacijski programer i sofisticirani korisnici) i administratorom operacijskog sustava

Arhitektura sustava za upravljanje bazama podataka

SUBP kao komponenta drugih sustava

Klijent-poslužitelj arhitektura

- *Client-server* (C/S)



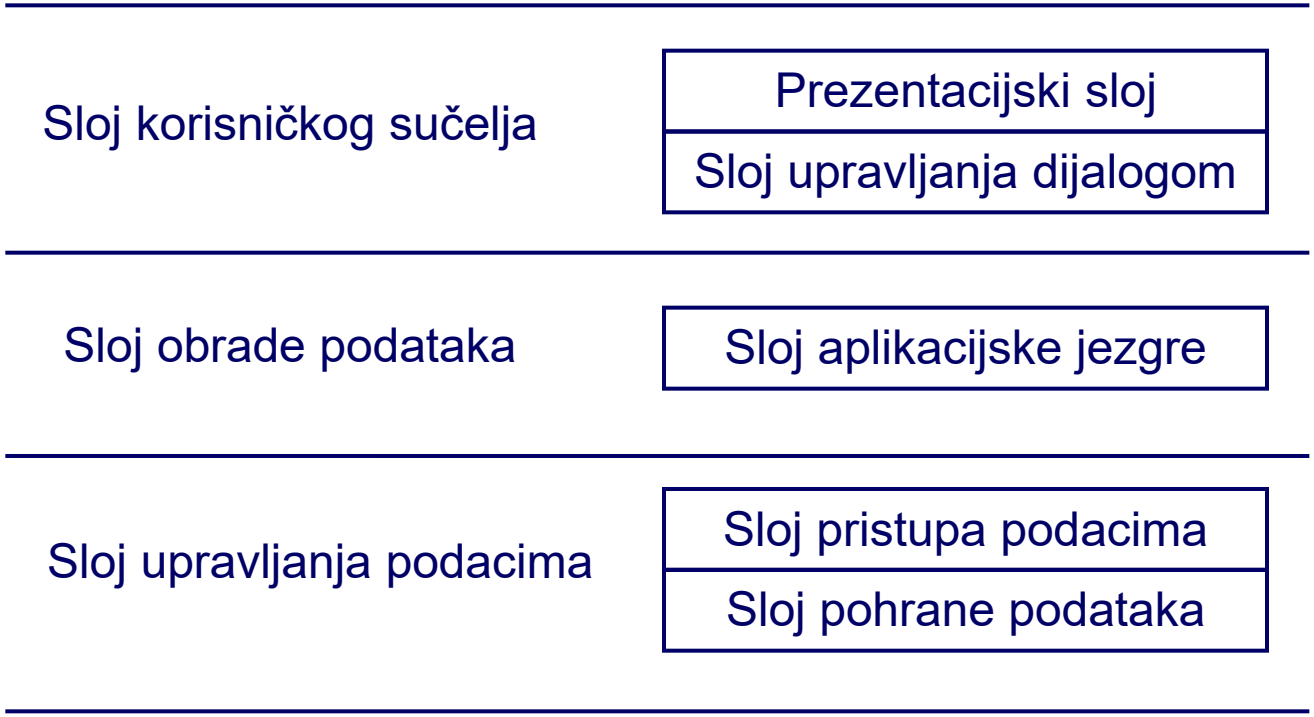
- sustav obuhvaća dvije komponente: klijent i poslužitelj
- koncept "zahtjev-odgovor" ("*request-response*"):
  - klijent postavlja zahtjev - poslužitelj odgovara
- komunikacija se odvija preko dobro definiranih sučelja: npr. TCP/IP i standardna sučelja namjenskih programa (Application Program Interface - API): ODBC (Open Database Connectivity), JDBC ("Java Database Connectivity"), OLE/DB (Object Linking and Embedding/Database), itd.

Klijent-poslužitelj arhitektura i distribuirani sustavi

- postupak raslojavanja (distribucija) sustava - razdioba sustava na slojeve, od kojih svaki sadrži dio funkcionalnosti ukupnog sustava
- okruženje je tipično heterogeno
  - različito sklopovlje (*hardware*) i programska oprema, često isporučena od strane različitih proizvođača
- slojevi imaju fundamentalno različite potrebe za računalnim resursima (brzina procesora, memorija, brzina i kapacitet diskova, U/I karakteristike)
  - npr. zahtjevi za sklopovljem i programskom potporom različiti su kod poslužitelja baze podataka i grafičke stanice

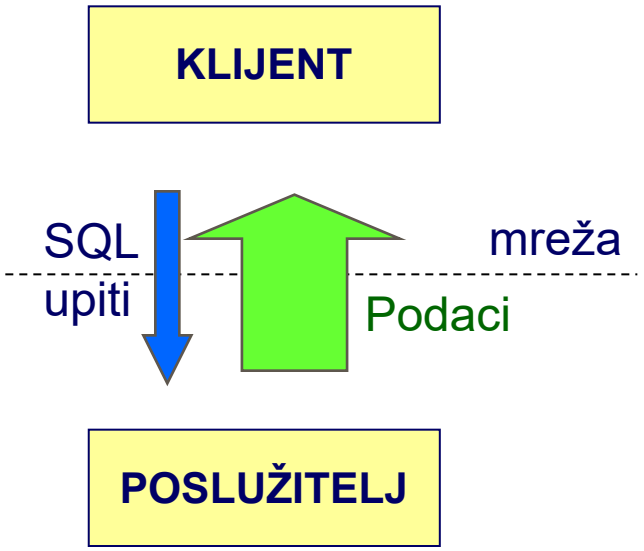
Klijent-poslužitelj arhitektura

- tri sloja i njihovi podslojevi
  - različitim načinom grupiranja slojeva/podslojeva u komponente dobivaju se različiti tipovi arhitektura



Dvoslojna (*two-tiered*) klijent-poslužitelj arhitektura oslonjena na klijenta (*client-centric*) - *fat client*, *thick client*

1. komponenta:
sloj korisničkog sučelja +
sloj obrade podataka



2. komponenta:
sloj upravljanja podacima

Client program:

```
...
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(
    "SELECT * FROM racun");
int suma = 0;
while(rs.next()) {
      int brRac = rs.getInt(1);
      int iznos = rs.getInt(2);
      if (provjeraKontrolneZnamenke(brRac)) {
         suma = suma + iznos;
      }
}
...
ustmt = conn.prepareStatement(
      "INSERT INTO zbirno VALUES(?, ?)");
ustmt.setDate(1, "4.3.2004");
ustmt.setInt(2, suma);
ustmt.executeUpdate();
...
```

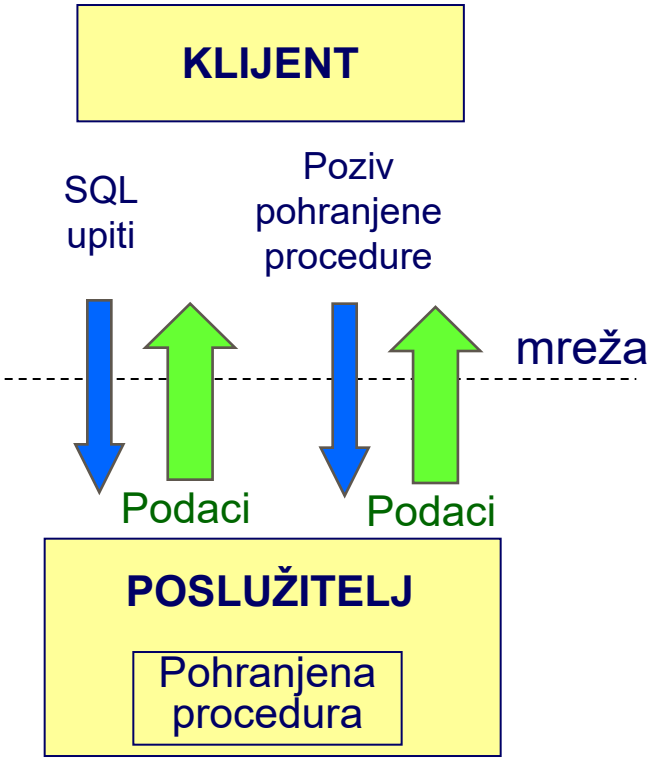
Dvoslojna (*two-tiered*) klijent-poslužitelj arhitektura oslonjena na klijenta (*client-centric*) - *fat client*, *thick client*

- korisničko sučelje (najčešće *GUI-graphical user interface*) usko je povezano s elementima za obradu podataka koji se u cijelosti implementiraju na klijentu

- Nedostaci:
  - *fat client sindrom* - budući da se obrada podataka obavlja na klijentu, klijent mora biti relativno snažno računalo
    - povećani troškovi opreme
    - degradacija performansi komunikacijske mreže, jer se podaci radi obrade moraju dobaviti na računalo-klijent
    - povećani troškovi održavanja programske potpore (promjene elemenata obrade podataka moraju se implementirati na svim računalima-klijentima)

Dvoslojna (*two-tiered*) klijent-poslužitelj arhitektura oslonjena na poslužitelja (*server-centric*) - *thin client*

1. komponenta:
sloj korisničkog sučelja



2. komponenta:
sloj za obradu podataka +
sloj upravljanja podacima

Client program:

```
...
Statement stmt = conn.createStatement();
stmt.executeUpdate(
    "EXECUTE PROCEDURE proc1()");
...
```

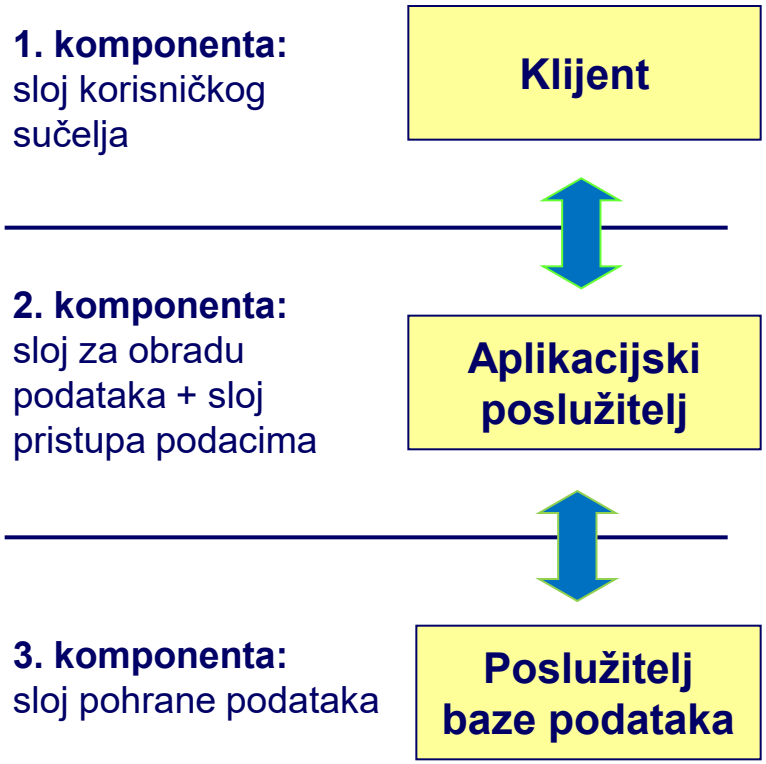
Stored procedure:

```
PROCEDURE proc1 ()
  DEFINE suma INTEGER; ...
  LET suma = 0;
  FOREACH
    SELECT brRac, iznos INTO
      pBrRac, pIznos FROM racun
     IF (provjeraKontrolneZnamenke(pBrRac)) THEN
        LET suma = suma + iznos;
     END IF
  END FOREACH
  INSERT INTO zbirno VALUES(TODAY, suma);
END PROCEDURE
```

Dvoslojna klijent-poslužitelj arhitektura oslonjena na poslužitelja (*server-centric*)

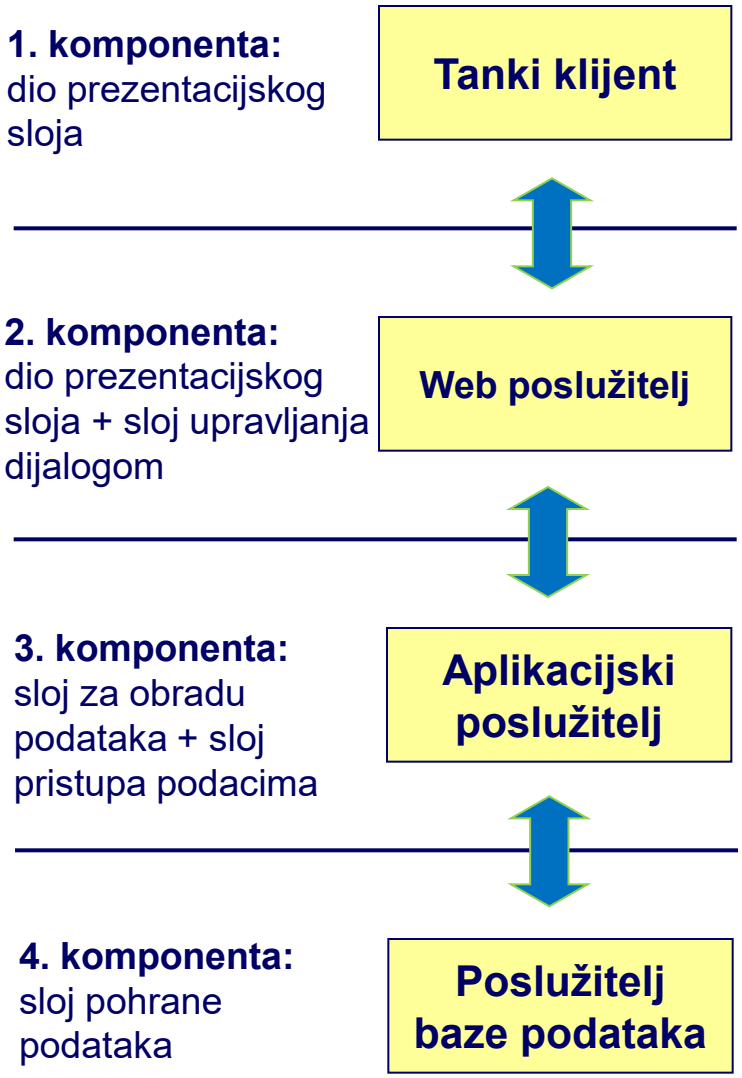
- Nedostaci:
  - poslužitelj može postati usko grlo - *bottle-neck*
  - najčešće se koristi poseban jezik za komponentu aplikacije na strani klijenta, poseban za komponentu aplikacije na strani poslužitelja
  - dio elemenata obrade podataka se ipak mora implementirati na klijentu, jer jezici koji se (za npr. pohranjene procedure) koriste na poslužitelju često nemaju sve mogućnosti koje imaju jezici koji se koriste u klijentskom sloju.

Troslojna (*three-tiered*) klijent-poslužitelj arhitektura



- primjer troslojne arhitekture:
  - klijent je zadužen za prezentaciju i upravljanje dijalogom: PC računalo s bogatim grafičkim korisničkim sučeljem (GUI) pisan u nekom prikladnom programskom jeziku (C#, C++, Java ...)
  - aplikacijski poslužitelj implementira sloj za obradu podataka i odgovoran je za upravljanje transakcijama, upravljanje opterećenjem sustava, sigurnost, itd. (BEA Tuxedo, IBM WebSphere Application server, Oracle Application Server, ...)
  - poslužitelj baze podataka je neki od sustava za upravljanje bazom podataka (Oracle, IBM DB2, IBM IDS, SQL Server...)

Višeslojna (*multi-tiered*) klijent-poslužitelj arhitektura

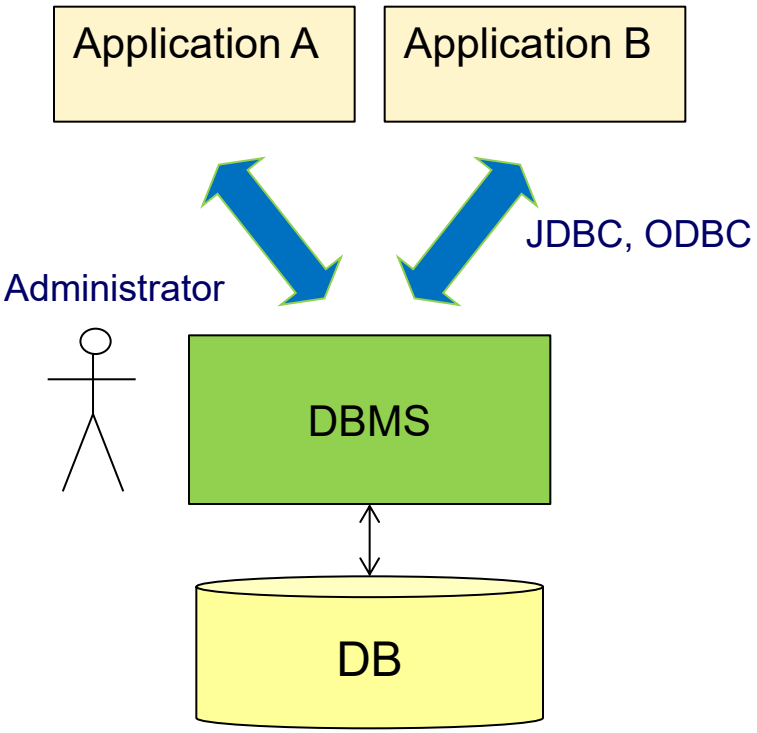


- primjer višeslojne arhitekture:
  - klijent je internet preglednik (*browser*) koji prezentira HTML stranice (ili mobilni uređaj, ...)
  - dio srednjeg sloja za upravljanje dijalogom je web poslužitelj podržan nekom od web tehnologija (JSP-Java ServerPages, ASP-Active Server Pages, ...)
  - dio srednjeg sloja s poslovnim pravilima je aplikacijski poslužitelj koji podržava neku od tehnologija (EJB-Enterprise JavaBeans, CORBA-Common Object Request Broker Architecture, ...)
  - poslužitelj baze podataka je neki od sustava za upravljanje bazom podataka (Oracle, IBM DB2, ...)

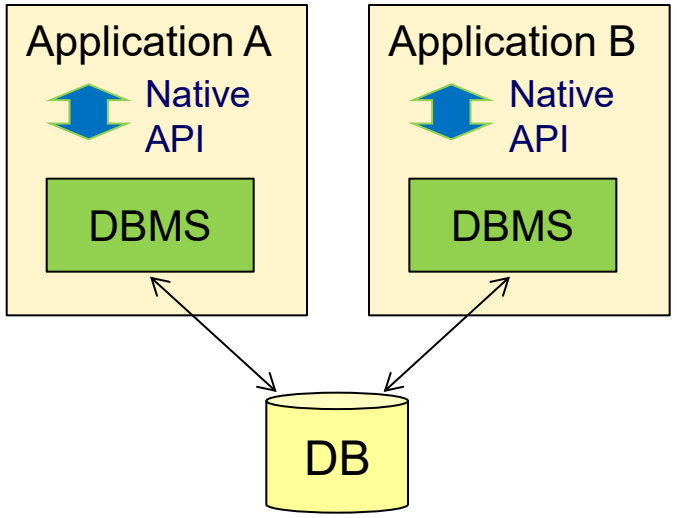
Ugrađeni sustavi za upravljanje bazama podataka

- *Embedded database management system*
  - Sustav za upravljanje bazama podataka integriran je u aplikaciju (*software product*)

"Enterprise DBMS"



"Embedded DBMS"



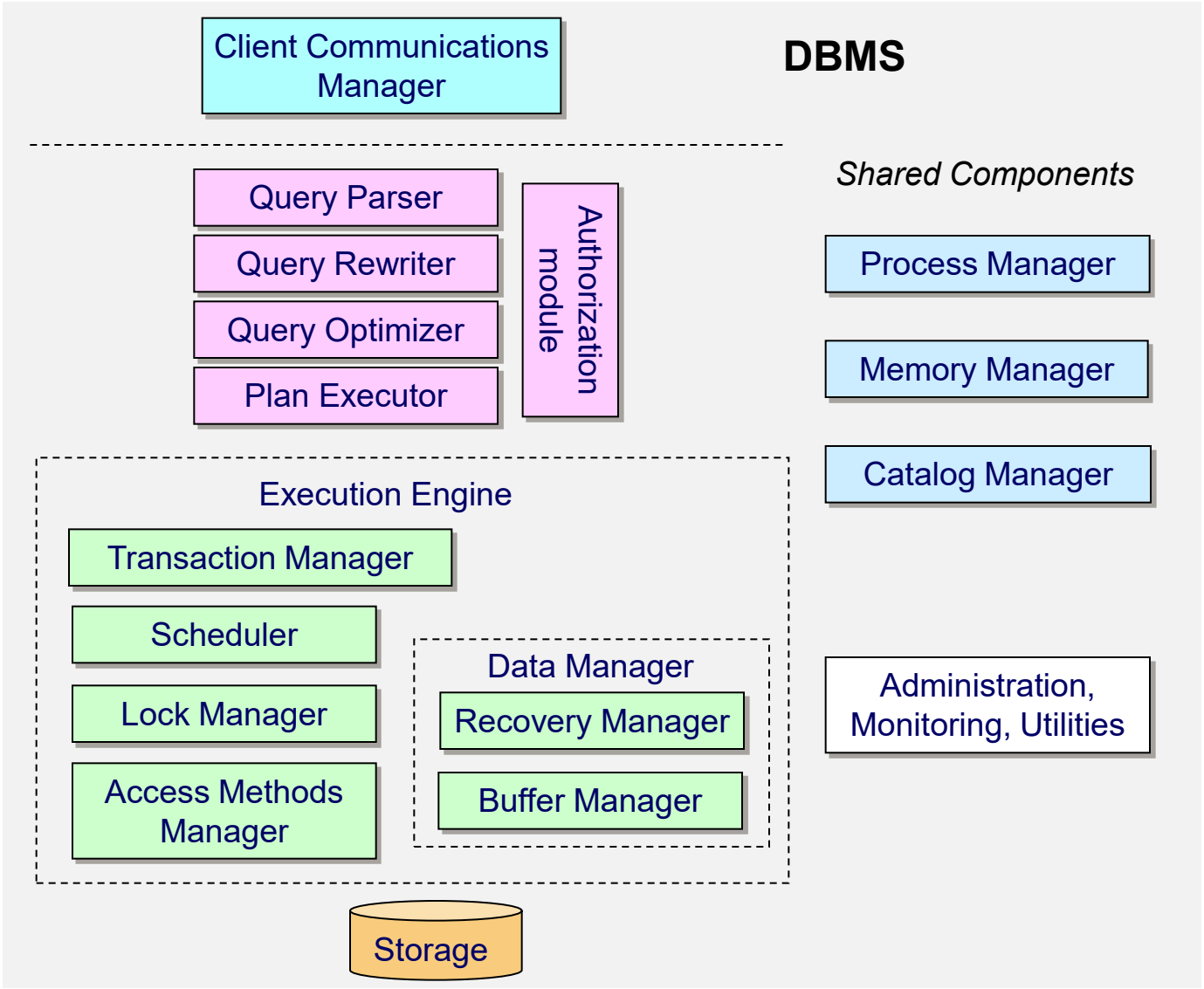
SUBP ugrađen u aplikaciju

- samoupravljiv (*self-managed DBMS*)
- nezahtjevan prema resursima (*small footprint*)
- modularan (*componentized DBMS*)

Komponente SUBP-a

Komponente SUBP-a

"The Life of a Transaction"



Komponente SUBP-a

- *"The Life of a Transaction"*

Uspostava komunikacije s korisnikom

- klijentska aplikacija (namjenski program) putem sučelja otvara korisničku sjednicu (*user session*) s menadžerom komunikacija (*Client Communications Manager*) čije su zadaće:
  - utvrditi sigurnosni status korisnika (npr. *login*, *password*)
  - uspostaviti i održavati vezu (pamtiti stanje veze) prema klijentskoj aplikaciji
  - prihvaćati zahtjeve, vraćati rezultate (podatke, kontrolne poruke)
- menadžer procesa (*Process Manager*) pokreće *dretvu izvršavanja* (*thread of execution)* koja će biti zadužena za obrađivanje zahtjeva vezanih uz dotičnu korisničku sjednicu
  - nazivi u literaturi: *server process* ili *DBMS worker*
  - za svaku korisničku sjednicu postoji jedan *DBMS worker*

Komponente SUBP-a

- *"The Life of a Transaction"* - nastavak

Prevođenje i optimizacija

- operacije transakcije (npr. SQL upiti) predaje se parseru (*Query Parser*) i modulu za pojednostavljivanje i normalizaciju upita (*Query Rewriter*)
- rezultat optimiranja upita u modulu za optimizaciju (*Query Optimizer*) je interni plan izvršavanja (*query plan*) kojim je određen "optimalni" redoslijed izvršavanja tipičnih operacija (spajanje, selekcija, projekcija, agregacija, sortiranje, itd.)
- interni plan izvršavanja predaje se izvršitelju plana (*Plan Executor*) koji implementira operacije spajanja, selekcije, itd. koristeći usluge modula na nižim razinama
- modul za autorizaciju (*Authorization Module*) može u svakoj od navedenih faza prevođenja i optimizacije biti konzultiran u vezi dozvola za izvršavanje pojedinih dijelova upita

Komponente SUBP-a

- *"The Life of a Transaction"* - nastavak

Izvršavanje

- modul za izvršavanje (*Execution Engine*) izvršava *transakcije*:
  - menadžer transakcija (*Transaction Manager*) objedinjuje pojedinačne zahtjeve (upite) u transakcije na temelju transakcijskih naredbi. Pojedinačne *database* operacije (*read, write*) i transakcijske (*transactional*) operacije (*start, abort, commit*) upućuje u menadžer redoslijeda izvršavanja
  - menadžer redoslijeda izvršavanja (*Scheduler*) uz pomoć menadžera zaključavanja (*Locking Manager*) određuje redoslijed izvršavanja operacija kojim se uz "*istodobno*" izvršavanje transakcija neće narušiti konzistentnost baze podataka
  - menadžer pristupa (*Access Manager*) koristi prikladne algoritme i strukture podataka za organiziranje i pristupanje podacima u sekundarnoj memoriji
  - menadžer međuspremnika (*Buffer Manager*) ima važnu ulogu osiguravanja efikasnog prijenosa podataka između primarne i sekundarne memorije
  - menadžer obnove (*Recovery Manager*) osigurava mogućnost obnove baze podataka u slučaju pogreške (kvara)

Komponente SUBP-a

- *"The Life of a Transaction"* - nastavak

Rezultati

- modul za izvršavanje dobivene rezultate smješta u međuspremnike menadžera komunikacija koji ih prosljeđuje klijentskoj aplikaciji
  - ako se radi o relativno velikim skupovima podataka klijentska aplikacija može dohvaćati dijelove rezultata, što će rezultirati iterativnim aktiviranjem metoda u menadžeru komunikacija i modulu za izvršavanje

Komponente SUBP-a

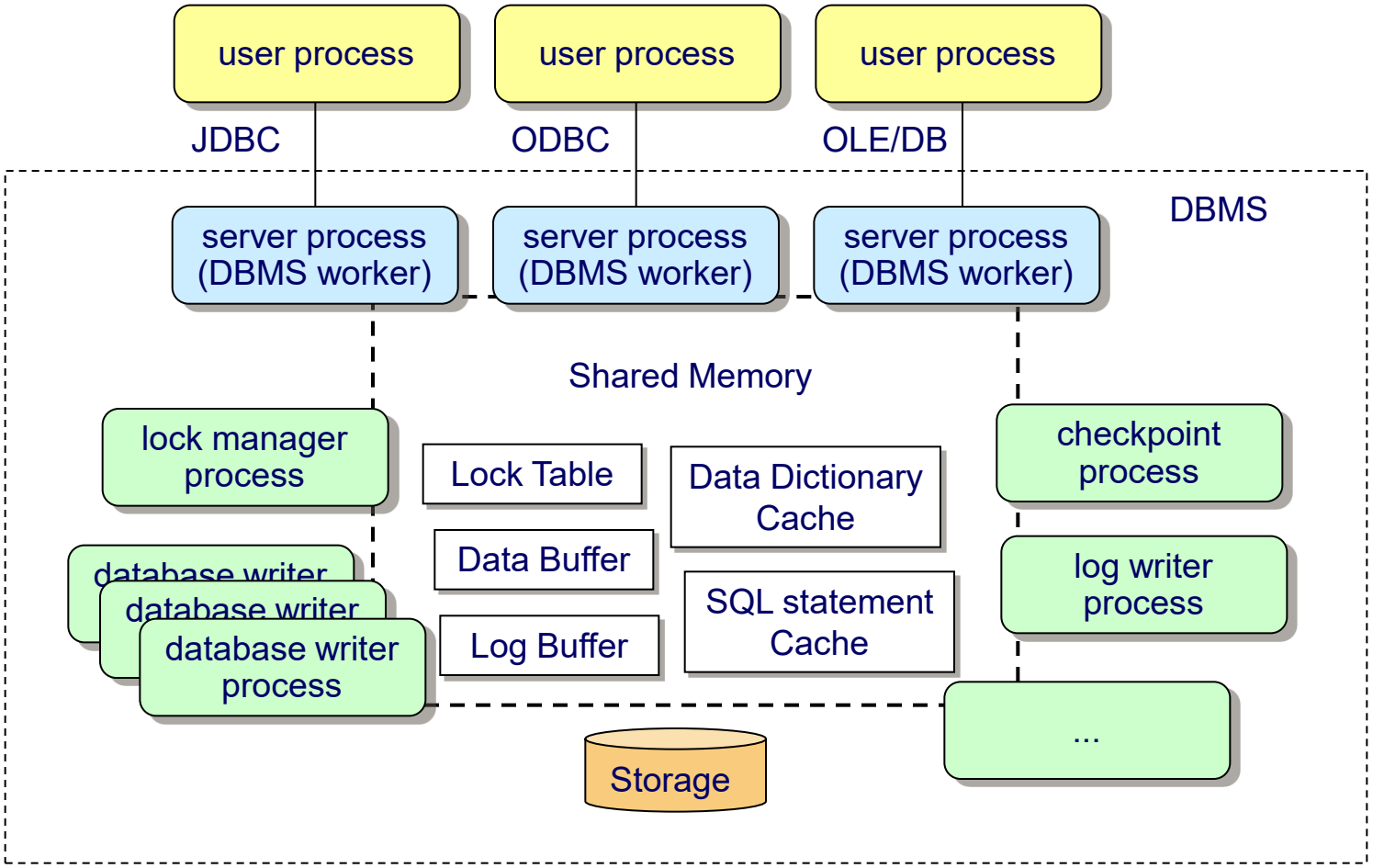
- *"The Life of a Transaction"* - nastavak

Zajedničke komponente (*Shared Components*)

- zajedničke komponente koriste se kao servisi do sada opisanih modula
  - menadžer procesa (*Process Manager*) pokreće, zaustavlja i sinkronizira procese
  - menadžer memorije (*Memory Manager*) upravlja alokacijom i dealokacijom primarne memorije
  - menadžer kataloga (*Catalog Manager*) osigurava meta-podatke koji se koriste za autorizaciju, parsiranje i optimizaciju upita
- SUBP obuhvaća i niz pomoćnih (ali važnih) modula (*Administration, Monitoring, Utilities*) za administraciju i nadgledanje sustava, npr.
  - modul za upravljanje arhiviranjem i obnovom
  - modul za promjenu konfiguracijskih parametara
  - modul za nadzor i upravljanje prostorom u postojanoj memoriji
  - modul za nadzor i upravljanje procesima
  - itd.

Procesi u SUBP-u

- Tipični sustav obuhvaća veliki broj procesa koji pristupaju podacima u dijeljenoj (*shared*) memoriji



Procesi u SUBP-u

Informativno

- *User process*
  - proces klijentske aplikacije (nije SUBP proces)

- *Server process (DBMS Worker)*
  - zaprima zahtjeve klijentskih aplikacija i vraća rezultate
  - upravlja korisničkom sjednicom (*user session*)
    - kontekst u kojem jedan korisnik obavlja niz SQL naredbi putem jedne veze (*SQL-Connection*) prema sustavu za upravljanje bazama podataka
- *Lock manager process*
  - implementira funkcionalnost menadžera zaključavanja
- *Database writer process*
  - jedan ili više procesa obavljaju prijenos podataka između primarne i sekundarne memorije
- *Log writer process*
  - upravlja zapisivanjem dnevnika u stabilnu memoriju
- *Checkpoint process*
  - periodički obavlja proces zapisivanja kontrolne točke

Organizacija procesa u SUBP-u

Informativno

- ***OS process per DBMS thread of execution***
- organizacije procesa u kojoj se svakoj dretvi izvršavanja (thread of execution, DBMS Worker, Server process) pridružuje jedan proces na razini operacijskog sustava (OS proces)
  - tip organizacije procesa korišten u ranijim implementacijama sustava
  - relativno jednostavna implementacija
    - koriste se mehanizmi operacijskog sustava
      - upravljanje procesima (*process switching*)
      - upravljanje dijeljenom memorijom, itd.
  - upitna skalabilnost (1000 korisnika $\rightarrow$ 1000 procesa)
    - povećanjem broja korisničkih sjednica raste broj OS procesa

Organizacija procesa u SUBP-u

Informativno

- danas sustavi u principu koriste organizaciju procesa u kojoj se dretva izvršavanja izvršava kao jedna od dretvi (*thread, lightweight process*) u okviru jednog OS procesa
  - jedan OS proces obavlja više dretvi
  - dretve dijele zajednički adresni prostor
  - trajanje zamjene konteksta dretve je znatno kraće od zamjene konteksta procesa
  - visoki stupanj skalabilnosti

- ***OS thread per DBMS thread of execution***
  - koristi se postojeći (opći) OS API za upravljanje dretvama
- ***DBMS thread per DBMS thread of execution***
  - SUBP koristi vlastiti (specijalizirani) API za upravljanje dretvama

Mediji za pohranu podataka

Hijerarhija medija za pohranu podataka

- Priručna memorija (*cache*)
- Glavna memorija (*main memory*)

} primarna memorija — RAM, DRAM

- Sekundarna memorija (online) — Flash-SSD, DRAM-SSD; magnetski disk (HDD); optički medij (CD, DVD)
- Tercijarna memorija (offline) — magnetska traka; *Tape silo/jukebox* sustavi (10^6 TB)

Karakteristike medija za pohranu podataka

- Priručna memorija (*cache*)
  - tipično SRAM
  - memorija u (ili pri) središnjoj procesorskoj jedinici (CPU), vrlo velike brzine pristupa podacima (~10 ns), vrlo skupa, vrlo ograničenog kapaciteta (~ MB)

- Glavna memorija (*main memory*)
  - tipično DRAM
  - velika brzina pristupa podacima (10-100 ns), relativno skupa, kapacitet (~ GB)
  - neprikladnost za (trajnu) pohranu baze podataka:
    - nepostojanost (gubitak napajanja, pogreška sustava)
    - cijena
    - kapacitet

Karakteristike medija za pohranu podataka

- Sekundarna memorija
  - postojana, sporija, većeg kapaciteta u odnosu na primarnu memoriju
- Magnetski disk (HDD)
  - kapacitet: do 16 TB (uz brzi napredak tijekom vremena)
  - osjetljivost na mehaničke udare (~ 300g)
  - vrijeme pristupa (*seek time* + *rotational latency*): 5-15 ms (uz spor napredak tijekom vremena)
  - brzina prijenosa podataka: 50-300 MB/s
    - *disk-to-buffer*, *buffer-to-computer*
    - manja za podatke s unutarnjih staza



Karakteristike medija za pohranu podataka

- karakteristike medija za pohranu podataka (postojanost, kapacitet, brzina pristupa podacima, cijena) uvjetuju da se većina današnjih baza podataka pohranjuje na magnetskim diskovima

- Sekundarna memorija (nastavak)
  - Flash-SSD
    - manji utrošak energije: ~ HDD/3
    - vrijeme pristupa podacima: ~ 0.1ms
    - brzina prijenosa podataka: ~ HDD
    - otpornost na vanjske utjecaje (-60/+90ºC, otpornost na udarce ~ 1500g)
    - nedostaci: cijena, ograničeni broj ciklusa pisanja ($3x10^5$ - 10^6)

Karakteristike medija za pohranu podataka

- Tercijarna memorija
  - sporija, većeg kapaciteta
    - za izradu sigurnosnih kopija (*backup*) ili za vrlo velike baze podataka
  - Optički mediji (CD, DVD)
    - kapacitet do ~ 17 GB
    - vrijeme pristupa podacima ~100 ms
  - Magnetske trake
    - kapacitet 40 GB - 15 TB
  - *Jukebox* ili *tape silo* (robotizirani sustavi)
    - za pohranu vrlo velikih baza podataka (100 TB - 1000 PB)
    - robotizirani sustav omogućava korištenje većeg broja jedinica medija (trake, CD, ...) u kombinaciji s manjim brojem uređaja za čitanje/pisanje
    - vrijeme pristupa: od nekoliko sekundi do nekoliko minuta

Karakteristike medija za pohranu podataka

- u kontekstu sustava za upravljanje bazama podataka treba razlikovati
  - **nepostojana (*volatile*) memorija**
    - sadržaj memorije se gubi u slučaju *pogreške sustava* (kvar napajanja, pogreška memorije, operacijskog sustava, ...)
    - SRAM, DRAM
  - **postojana (*non-volatile*) memorija**
    - sadržaj memorije ostaje sačuvan u slučaju pogreške sustava
    - međutim, pogreške medija (npr. *disk crash*) će ipak izazvati gubitak sadržaja
    - HDD, Flash-SSD, DRAM-SSD, traka, DVD, ...
  - **stabilna (*stable*) memorija**
    - sadržaj memorije se *nikad** ne gubi

* garancija za "nikad" ne postoji, ali moguće je postići razumnu razinu zaštite

Stabilna memorija

- implementacija: replikacijom podataka na medijima s postojanom memorijom

- najčešće: diskovi organizirani u RAID sustave - *on-line* stabilna memorija
  - *hot-pluggable*, *hot-spare*: eliminira se potreba za zaustavljanjem sustava u slučaju kvara medija
  - RAID ne garantira "stabilnost" memorije npr. u slučaju požara
- korištenje trake, CD, DVD (napraviti kopiju i pohraniti medij u drugom gradu)
  - pogodno za arhivske kopije (*backup*), ali ne omogućuje kontinuiranu zaštitu sadržaja: *off-line* stabilna memorija
- korištenje SAN (*Storage Area Network*) arhitekture
  - polje RAID organiziranih diskova fizički odvojenih od poslužitelja
    - povezivanje SCSI ili Fibre Channel sučeljem

Organizacija podataka u sekundarnoj memoriji

- logički zapisi (n-torke) pohranjuju se u blokove (fizičke zapise, stranice) sekundarne memorije
  - *block, physical record*, *page*
  - veličina bloka ovisi o sklopovlju, operacijskom sustavu, primijenjenom tipu datotečnog sustava (tipično: 512 B, 1 kB, 2 kB, 4 kB, 8 kB)
    - u jedan blok najčešće je smješteno više logičkih zapisa (npr. n-torki relacije)

Primjer:

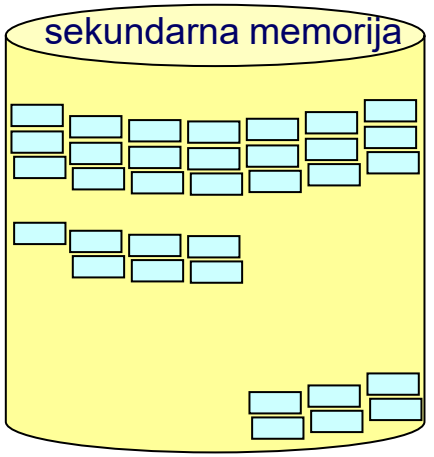
mbr	ime	prez
1999999	Ivan	Novak
1000315	Ana	Horvat
...		
1999998	Jura	Kolar
1000001	Tea	Ban

56 B

10^6 n-torki

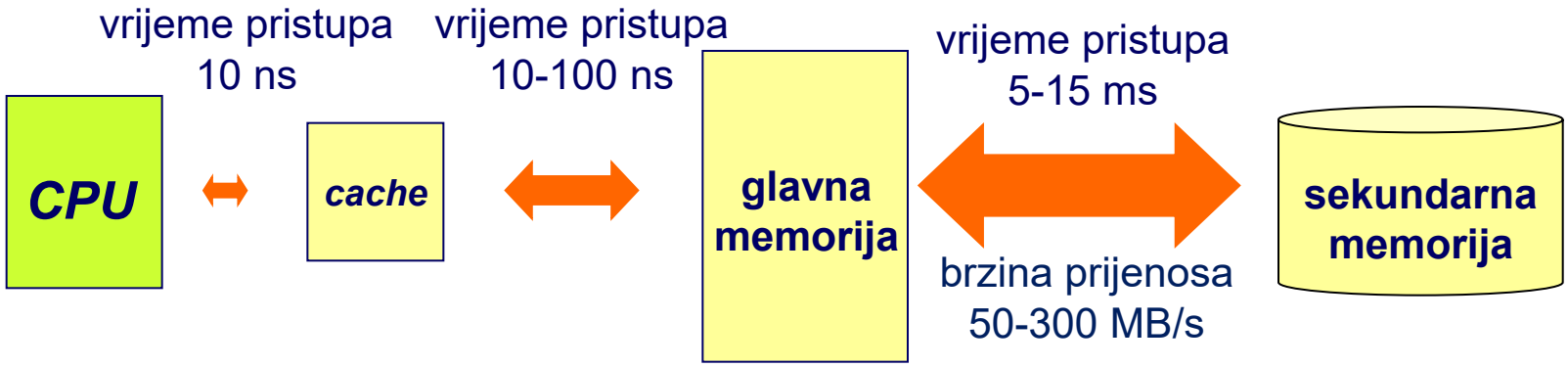
⇒ uz pretpostavku veličine bloka 1kB

⇒ broj potrebnih blokova u sekundarnoj memoriji: najmanje **54688**



Pristup podacima u sekundarnoj memoriji

- koliko je podatak "daleko od procesora"



- operacije čitanje/pisanje
  - *read, write*
- U/I - ulazno/izlazne operacije
  - obavljaju se u jedinicama blokova
  - *input, output (I/O)*
  - *disk-read, disk-write*

- zbog velike razlike u brzini (~ ns : ~ ms)

  ⇒ **dominiraju troškovi U/I operacija**

Utjecaj broja U/I operacija na performanse sustava

- ukupno vrijeme potrebno za prijenos blokova iz sekundarne u primarnu memoriju najviše ovisi o sljedećim karakteristikama medija
  - *seek time*
  - *rotational delay (latency)*
  - *transfer time*
- znatan utjecaj ima stupanj fragmentacije (koliko su blokovi koji se prenose raspršeni po stazama/cilindrima i sektorima na disku)

Primjer:

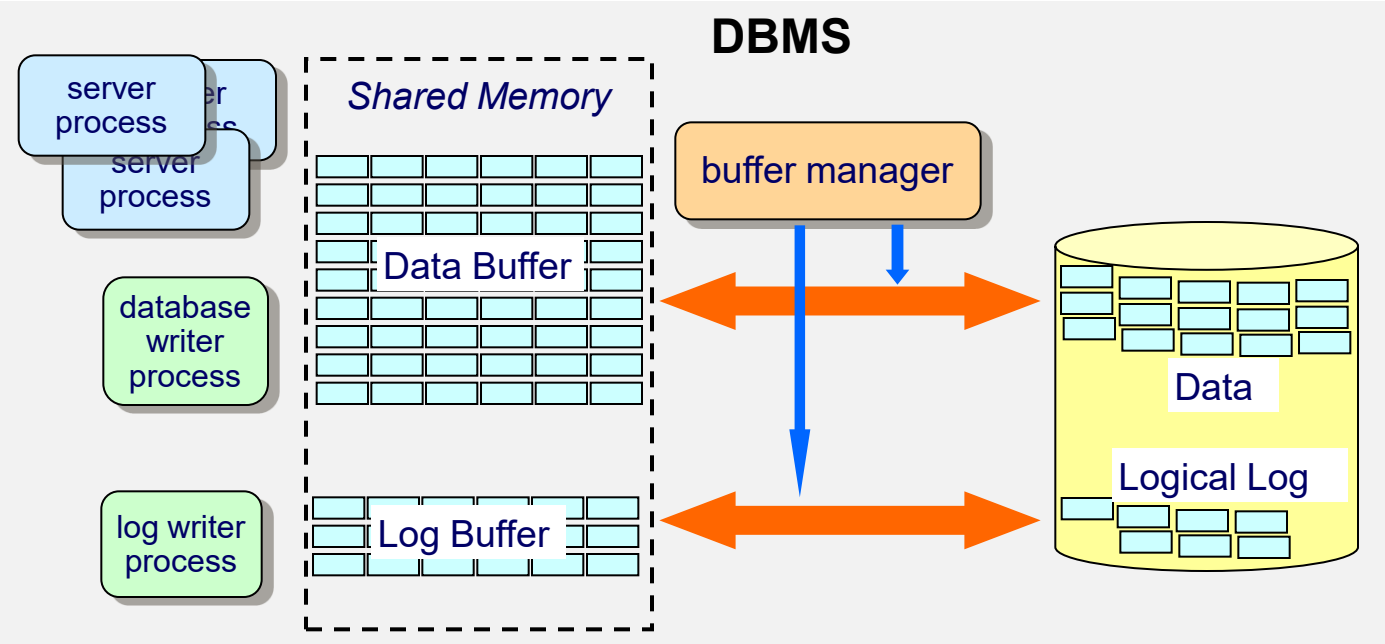
- neka se nad podacima iz prethodnog primjera obavlja upit

```
SELECT * FROM osoba
  WHERE mbr BETWEEN 1222000 AND 1222300;
```

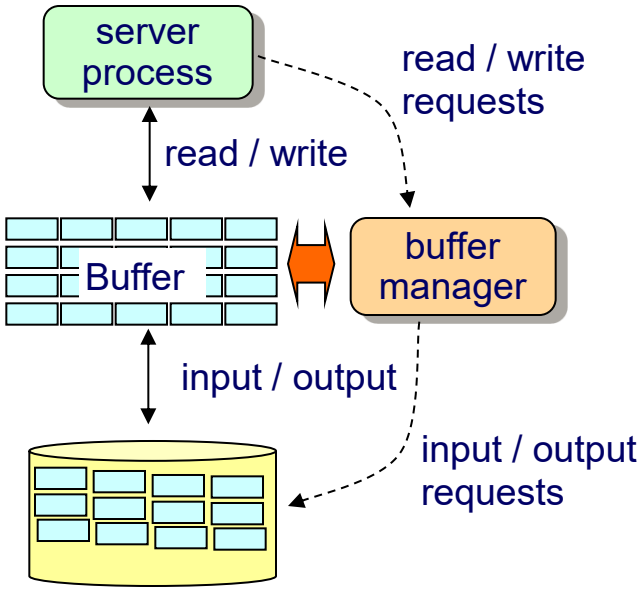
- uz pretpostavku korištenja slijednog pretraživanja, svaki od 54688 blokova mora se prenijeti u glavnu memoriju i pretražiti
  - za prijenos podataka očekuje se utrošak vremena mjeren sekundama
  - za pretraživanje svih blokova u glavnoj memoriji očekuje se utrošak vremena mjeren milisekundama

Organizacija podataka u glavnoj memoriji

- međuspremnik (*buffer* ili *buffer pool*)
  - raspoloživa primarna memorija particionirana je u blokove koji svojom veličinom u pravilu odgovaraju blokovima u sekundarnoj memoriji
- komponente SUBP-a koriste sekundarnu memoriju posredstvom menadžera međuspremnika



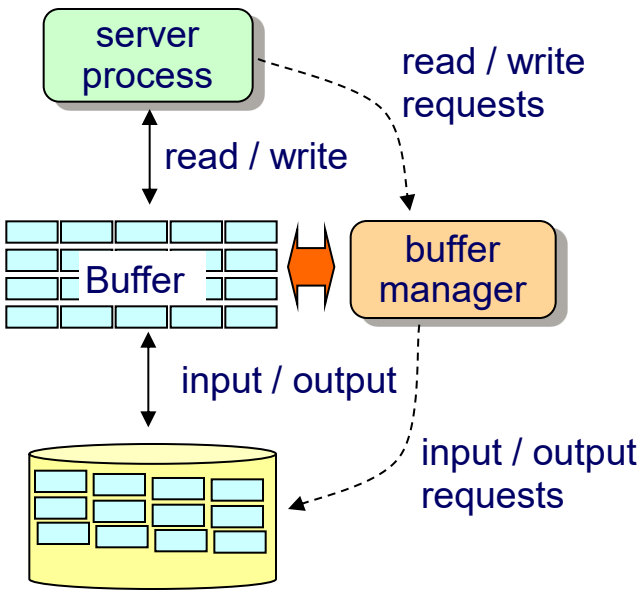
Uloga menadžera međuspremnika (za operaciju read)



- proces koji treba obaviti operaciju čitanja postavlja *read* zahtjev menadžeru međuspremnika
- ako se traženi blok već nalazi u međuspremniku, obavlja se operacija read
- inače, menadžer prethodno mora obaviti operaciju *input* kojom se traženi blok iz sekundarne memorije kopira u međuspremnik
  - iskorišteni blok može ostati u međuspremniku

- blokove je poželjno čim dulje zadržati u međuspremniku, međutim u međuspremniku ne mogu ostati zauvijek zbog ograničenog kapaciteta međuspremnika

Uloga menadžera međuspremnika (za operaciju write)



- proces koji treba obaviti operaciju pisanja postavlja *write* zahtjev menadžeru međuspremnika
- ako se traženi blok već nalazi u međuspremniku, obavlja se operacija *write*
- inače, menadžer prethodno mora obaviti operaciju *input* kojom se traženi blok iz sekundarne memorije kopira u međuspremnik
  - iskorišteni blok može ostati u međuspremniku, ali prije ili kasnije morat će se obaviti operacija *output*, svakako prije trenutka u kojem će se taj blok (*dirty page*) izbaciti iz međuspremnika
  - dakle, operacija *output* se ne mora (uvijek) obaviti odmah nakon *write*
    - *vidjet ćemo u poglavlju o obnovi baze podataka*

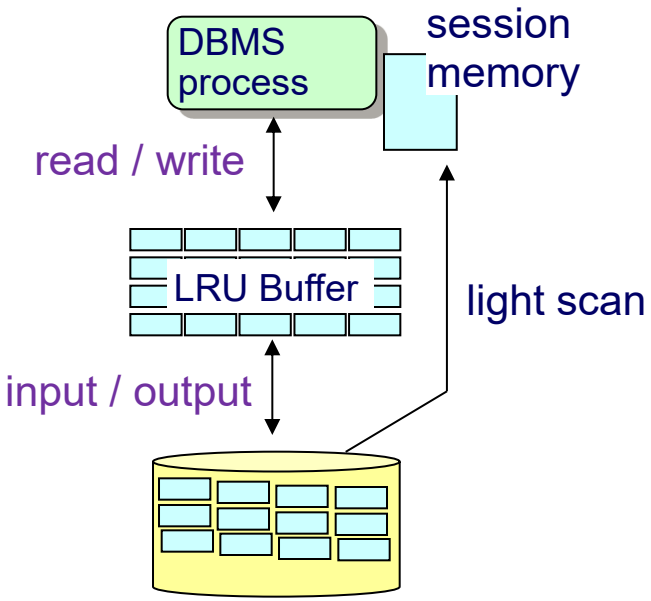
Strategije zamjene stranica

- algoritam koji određuje koje blokove (i kada) izbaciti iz međuspremnika (*buffer-replacement policy*) ima veliki utjecaj na efikasnost sustava
  - Least-Recently Used (LRU) - nisu korišteni tijekom najduljeg vremena
  - Most-Recently Used (MRU) - proteklo najmanje vremena od zadnjeg korištenja
  - First-In-First-Out (FIFO)
  - Clock Algorithm
- u današnjim sustavima najčešće se koristi LRU algoritam*
  - pretpostavka: blok kojeg je neki proces koristio nedavno (*recently*) vjerojatno će uskoro opet koristiti isti ili neki drugi proces
  - iz međuspremnika izbacivati (po potrebi uz operaciju *output*) blokove koje tijekom najduljeg perioda nije koristio niti jedan proces
  - menadžer međuspremnika mora za svaki blok u međuspremniku evidentirati zadnji trenutak u kojem je neki proces koristio taj blok

* u nekim slučajevima se operacija *output* mora obaviti "odmah", bez obzira na LRU algoritam
→ u poglavlju o obnovi baze podataka

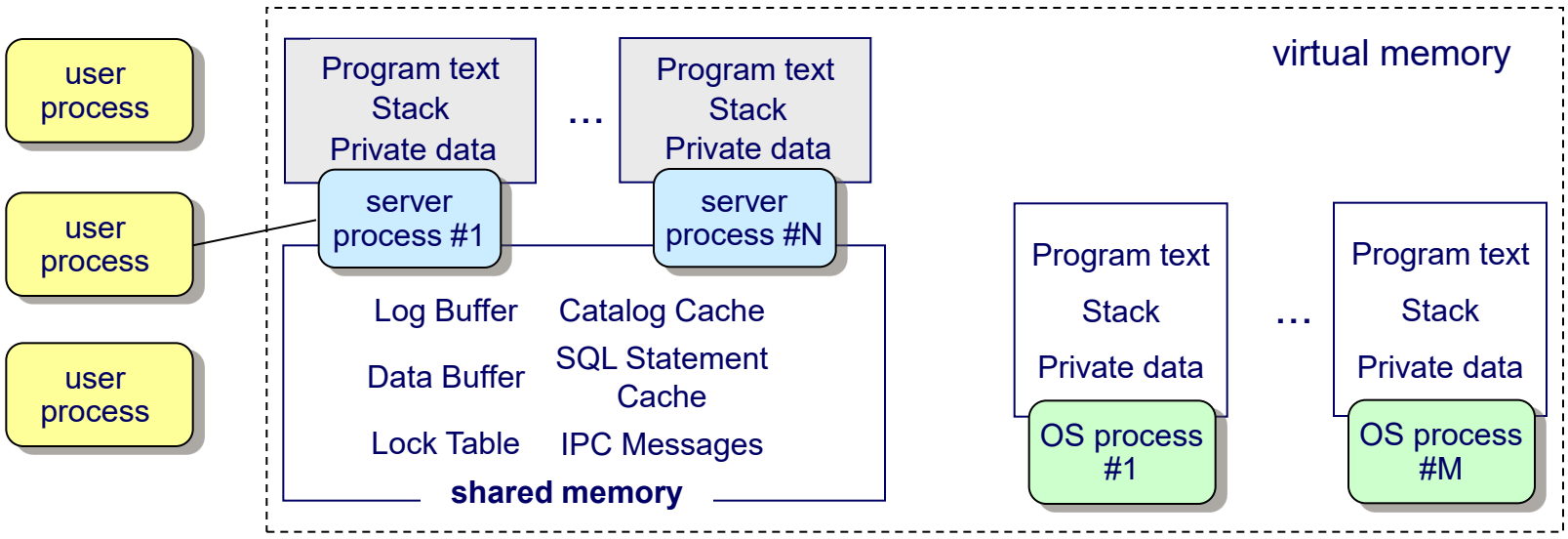
LRU - kada se ne koristi?

- slijedno čitanje relacije čija je veličina sumjerljiva ili veća od raspoložive veličine međuspremnika. Npr. čitanje relacija od 4 500 blokova, uz raspoloživu veličinu međuspremnika od 5 000 blokova
  - posljedica: pročitane stranice (koje se u međuspremniku trebaju nalaziti tek kratko vrijeme) nepotrebno uzrokuje izbacivanje svih drugih, vjerojatno često korištenih stranica
  - Oracle rješenje: koristi uobičajeni (LRU) međuspremnik, ali se tom operacijom pročitane stranice smjesta označavaju kao *least recently used* stranice, odnosno kao stranice koje dugo vremena nisu korištene
  - IBM Informix rješenje: *light scans*: međuspremnik se zaobilazi i podaci se iz sekundarne memorije kopiraju izravno u memoriju koja je u tu svrhu alocirana za korisničku sjednicu u kojoj se izvršava ta operacija



Uloga dijeljene memorije (*shared memory*)

- dijeljena memorija je mehanizam operacijskog sustava koji iz perspektive SUBP-a omogućava:
  - smanjenje broja U/I operacija
    - upravljanje međuspremnicima na temelju zajedničkih potreba svih procesa, umjesto na temelju svakog procesa pojedinačno
  - visoku brzinu komunikacije među procesima i dretvama (IPC - *inter-process communication*)
    - razmjena poruka među procesima se obavlja brzinom prijenosa u memoriji, nasuprot sporijim mehanizmima, npr. cjevovodima (*pipes*)



Utjecaj mehanizama operacijskog sustava

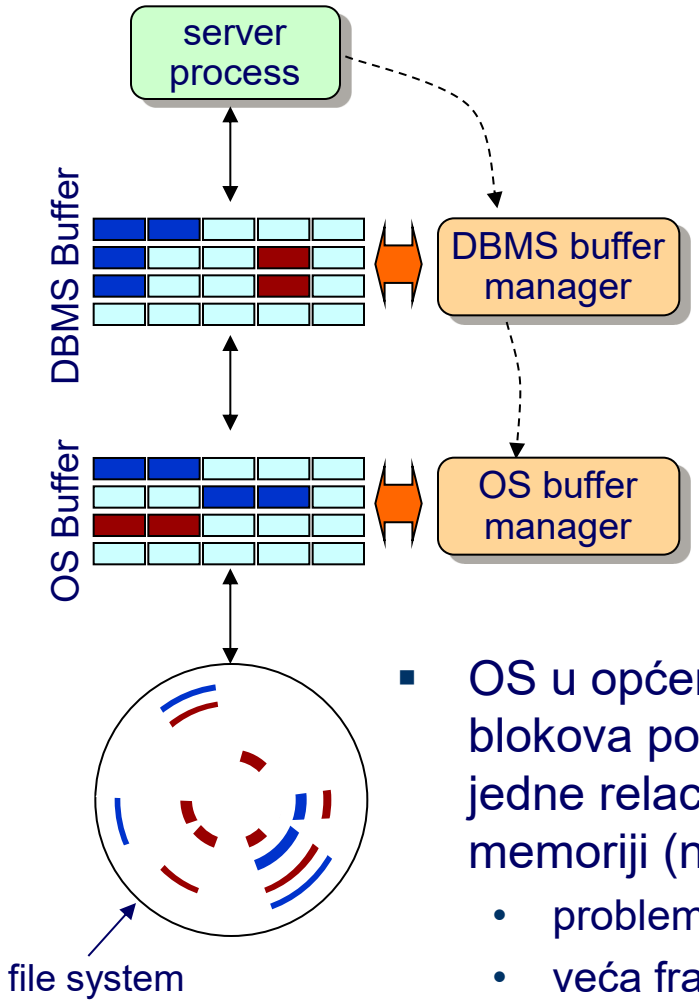
- operacijski sustav koristi vlastite mehanizme za organizaciju podataka i prijenos podataka između radnog spremnika i sekundarne memorije
  - podaci su u sekundarnoj memoriji organizirani u datotečni sustav (*file system*)
  - prenose se u blokovima, uz korištenje međuspremnika operacijskog sustava (često LRU algoritam) s ciljem poboljšanja performansi U/I operacija
- za SUBP koji za pohranu podataka u sekundarnoj memoriji koristi datotečni sustav i procedure operacijskog sustava (*system calls*) kaže se da koristi *cooked files* ili *buffered files*

- problemi:
  - korištenje dvostrukih međuspremnika, *double buffering*
  - moguće je da će logički kontinuirani podaci biti smješteni u fizički diskontinuiranom prostoru (fragmentacija relacije u fizičkom prostoru)

Double buffering

- korištenje međuspremnika operacijskog sustava u kombinaciji s međuspremnicima SUBP-a je štetno iz barem tri razloga:
  - temeljem podataka iz rječnika podataka, SUBP upravlja međuspremnicima *optimalno* u odnosu na operacije koje obavlja. Operacijski sustav nema tu mogućnost, pa svojim međuspremnicima upravlja manje efikasno od SUBP.
  - primarna memorija se nepotrebno troši za dvije kopije podataka
  - procesorsko vrijeme se nepotrebno troši na kopiranje podataka, i to u dva smjera
    - pisanje
      - međuspremnik SUBP $\rightarrow$ međuspremnik OS $\rightarrow$ sekundarna memorija
    - čitanje
      - sekundarna memorija $\rightarrow$ međuspremnik OS $\rightarrow$ međuspremnik SUBP

Cooked files



- korištenje međukoraka: disk ⇔ OS međuspremnici ⇔ SUBP međuspremnici ⇒ lošije performanse
- trošenje resursa s vrlo malim ili nikakvim dobitkom
- korištenje procedura opće namjene (operacijski sustav) za visoko specijalizirane zadaće koje obavlja SUBP ⇒ lošije performanse

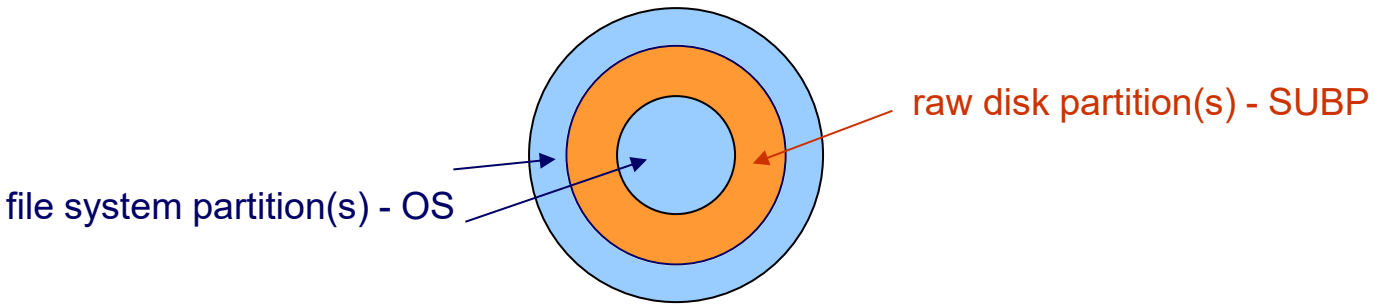
- OS u općem slučaju ne osigurava pohranu logički kontinuiranih blokova podataka (npr. blokova podataka koji sadrže n-torke jedne relacije) u fizički kontinuirani prostor u sekundarnoj memoriji (npr. u nekoliko susjednih traka)
  - problem: *seek time (naročiti problem)*, *rotational latency*
  - veća fragmentacija ⇒ sporiji pristup podacima

Kako SUBP može zaobići te nedostatke?

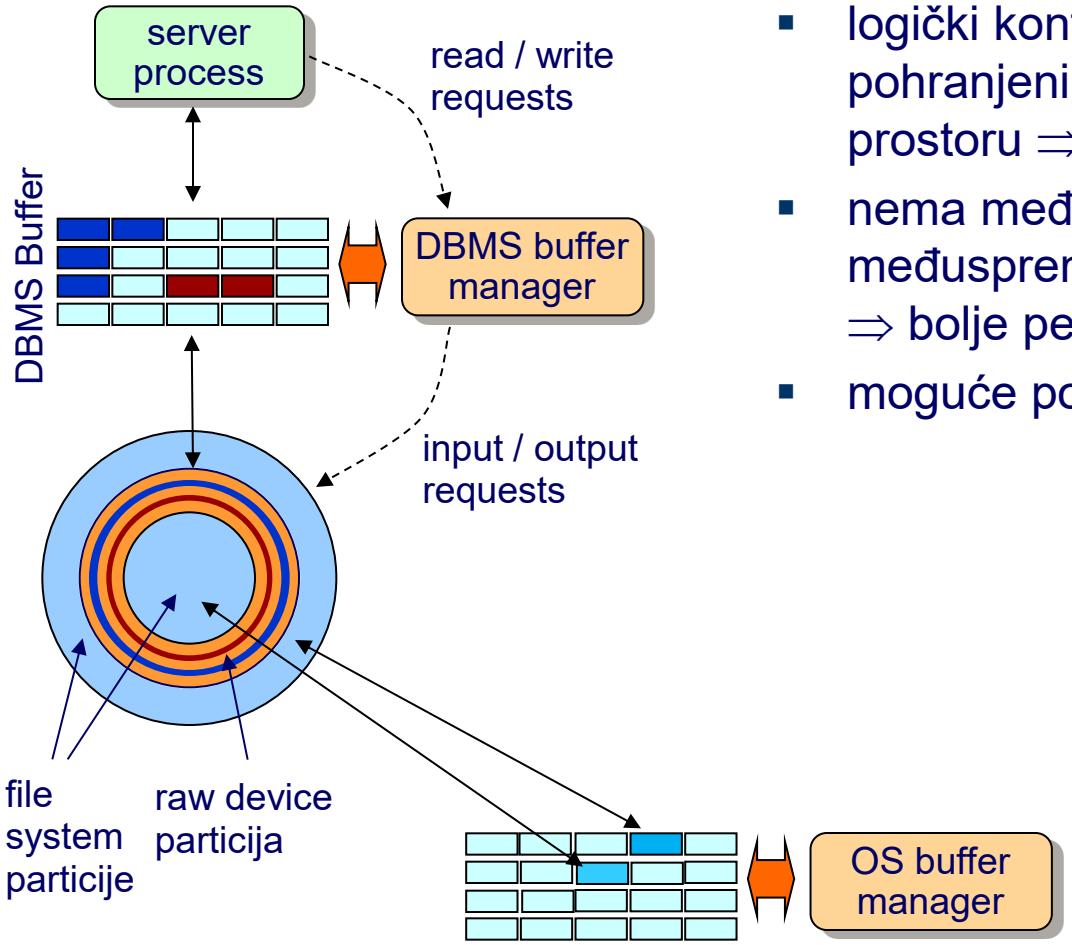
Raw disk I/O

- termin *raw disk* I/O odnosi se na način pristupa podacima u sekundarnoj memoriji: podacima se pristupa direktno, umjesto putem datotečnog sustava. Operacijski sustav omogućuje *raw disk I/O* putem posebnog sučelja
  - *raw disk device*
- korištenjem *raw disk I/O* SUBP

1. zaobilazi se međuspremnik i mehanizmi straničenja (*paging*) operacijskog sustava. SUBP u potpunosti upravlja procesima prijenosa podataka. Npr. blokovi će se u sekundarnu memoriju zapisivati točno onda kada za to postoji potreba iz perspektive SUBP-a
2. particija sekundarne memorije za koju se koristi *raw disk device* predstavlja fizički kontinuirani prostor



Raw disk I/O



- logički kontinuirani blokovi podataka pohranjeni su u fizički kontinuiranom prostoru ⇒ bolje performanse
- nema međukoraka: disk ⇔ OS međuspremnici ⇔ SUBP međuspremnici ⇒ bolje performanse
- moguće poboljšanje performansi 10-30%

Direct I/O

- danas se mehanizam raw disk rjeđe koristi jer operacijski sustavi pomoću mehanizma ***direct I/O*** omogućuju korištenje direktnog pristupa memoriji u datotečnom sustavu (tj. zaobilaženje međuspremnika operacijskog sustava)

- performanse sustava za upravljanje bazama podataka kada koristi mehanizam ***direct I/O*** uglavnom su sumjerljive performansama sustava koji koristi mehanizam *raw disk*

Upravljanje prostorom za pohranu u SUBP-u

- različiti SUBP omogućuju različite razine mogućnosti upravljanja prostorom za pohranu
- ovdje su na primjeru sustava IBM Informix prikazani temeljni koncepti

Fizičke jedinice pohrane

- jedinice koje predstavljaju segmente sekundarne memorije gledano iz perspektive operacijskog sustava
  - grumen (*chunk*)
  - područje (*extent*)

Logičke jedinice pohrane

- jedinice koje predstavljaju logički povezane dijelove baze podataka (iz perspektive sustava za upravljanje bazama podataka). Npr.
  - prostor baze podataka (*database space*)

- logičke jedinice pohrane se na razini SUBP-a mogu povezati s fizičkim jedinicama pohrane. Na taj način administrator SUBP-a može utjecati na fizički smještaj logičkih elemenata baze podataka

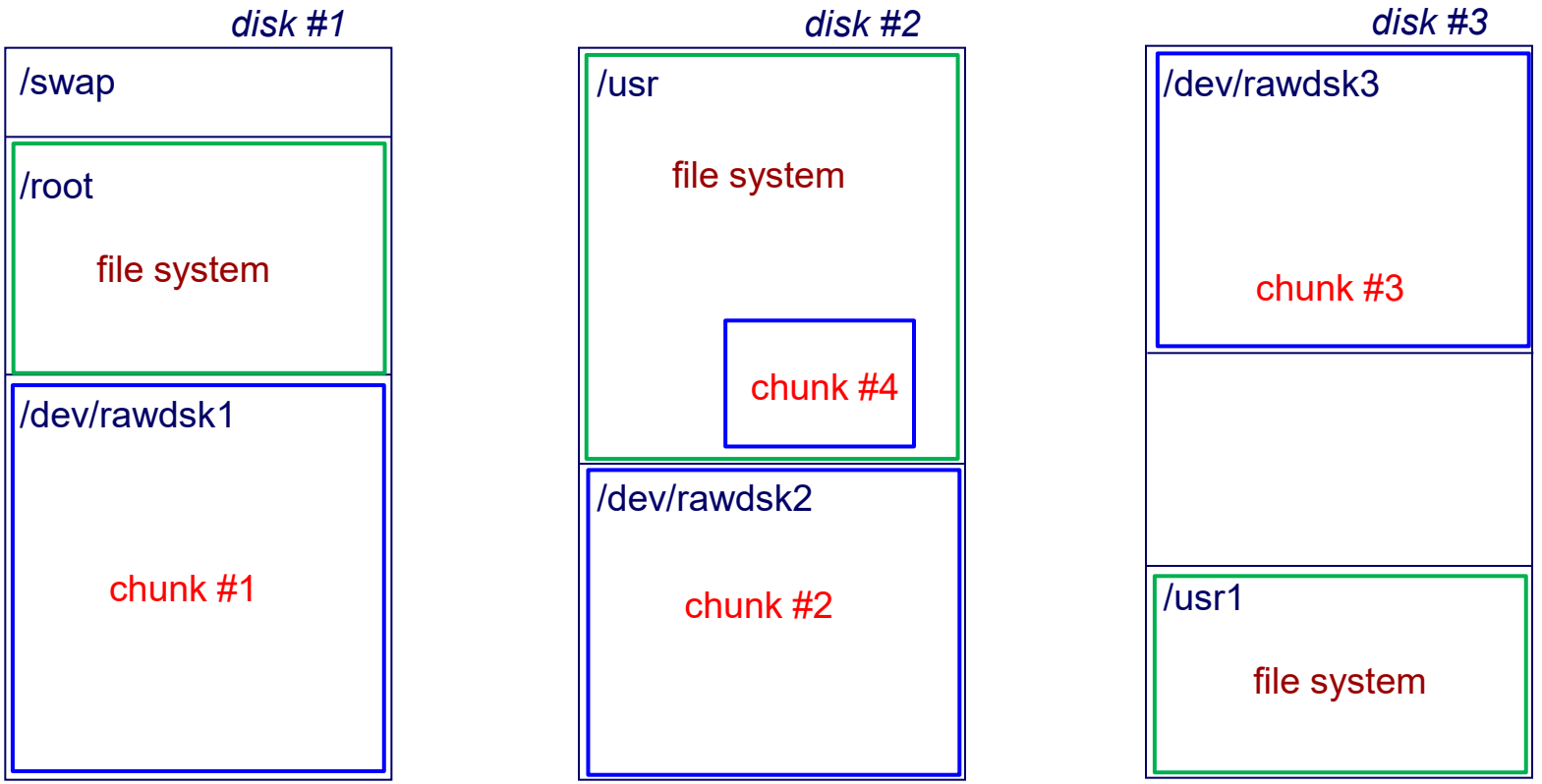
Fizičke jedinice pohrane

Grumen (*chunk*)

- grumen je najveća jedinica fizičkog prostora koja se s razine operacijskog sustava dodjeljuje sustavu za upravljanje bazama podataka. Predstavlja unaprijed alocirani prostor, u *raw device* obliku ili u obliku datoteke u datotečnom sustavu (*cooked file*).
- grumene definira administrator operacijskog sustava u dogovoru s administratorom sustava baza podataka (uobičajeno prilikom instalacije operacijskog sustava ili ugradnje novog diska)
  - particije diska koje će se koristiti kao *raw device*
  - datoteke koje će se koristiti kao *cooked file*

Grumen (*chunk*)

Primjer:



- fizički kontinuirani prostor u grumenima 1, 2 i 3
- grumen 4 (datoteka) nije nužno fizički kontinuirani prostor
- dodavanje novih grumena je moguće samo na slobodnim particijama, na diskovima koji se naknadno dodaju ili u datotečnom sustavu u formi datoteka

Područje (*extent*)

Područje (extent)

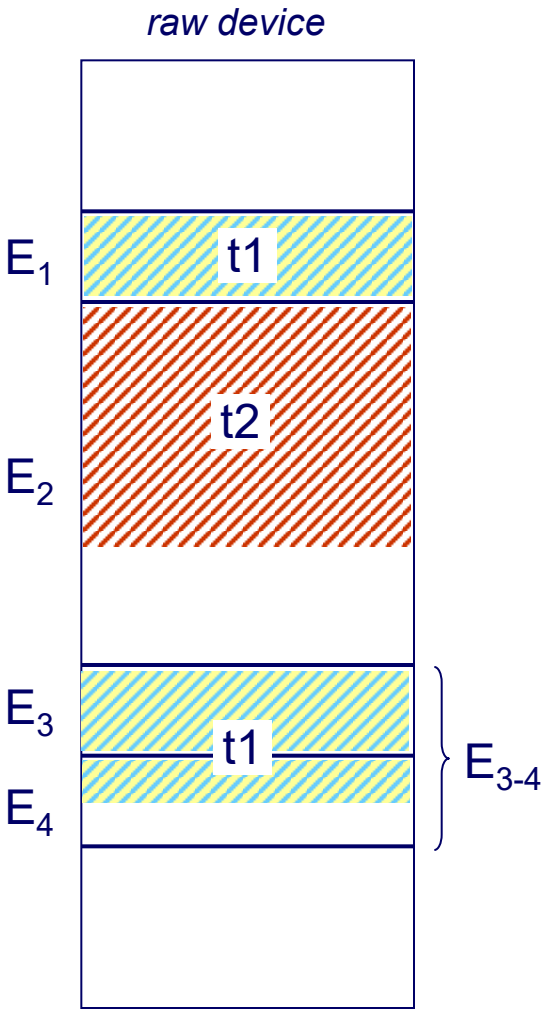
- SUBP nastoji zadržati podatke iste relacije u kontinuiranom prostoru sekundarne memorije
- u trenutku kreiranja relacije, SUBP rezervira fiksnu količinu prostora u sekundarnoj memoriji. Kada se taj inicijalno rezervirani prostor popuni, SUBP će alocirati sljedeći dio sekundarne memorije. Fizička jedinica prostora koja se rezervira u trenutku kreiranja relacije ili kasnije tijekom njenog proširenja, naziva se područje (*extent*)
- ako se koristi *raw device*, jedno područje je uvijek kontinuirani segment sekundarne memorije, inače, područje može biti više ili manje fragmentirano

```
CREATE TABLE ... (
    ...
) EXTENT SIZE m NEXT SIZE n;
```

- **m** i **n** su veličine područja izražene u kB
  - **m** veličina inicijalnog područja kod kreiranja relacije
  - **n** inkrementalna veličina dodatnog područja kod proširenja prostora relacije

Područje (*extent*)

Primjer:



CREATE TABLE t1 (...)
EXTENT SIZE 16 NEXT SIZE 16
⇒ alocira se E_1 = 16k

CREATE TABLE t2 (...)
EXTENT SIZE 64 NEXT SIZE 32
⇒ alocira se E_2 = 64k

INSERT INTO t1 (40 kB podataka)
⇒ popunjava se E_1 (16 kB)
⇒ alocira se E_3 (16 kB)
⇒ popunjava se E_3 (16 kB)
⇒ alocira se E_4 (16 kB)
⇒ E_3 i E_4 se spajaju u E_{3-4}
⇒ popunjava se dio E_{3-4} (8 kB)

INSERT INTO t2 (40 k podataka)
⇒ popunjava se dio E_2 (40 kB)

Područje (*extent*)

- **CILJ:** optimirati između
  - nepotrebnog zauzeća prostora
  - broja područja koja se pojedinačno alociraju

- ako broj fragmenata neke relacije postane prevelik, moguće je obaviti defragmentaciju

Logičke jedinice pohrane

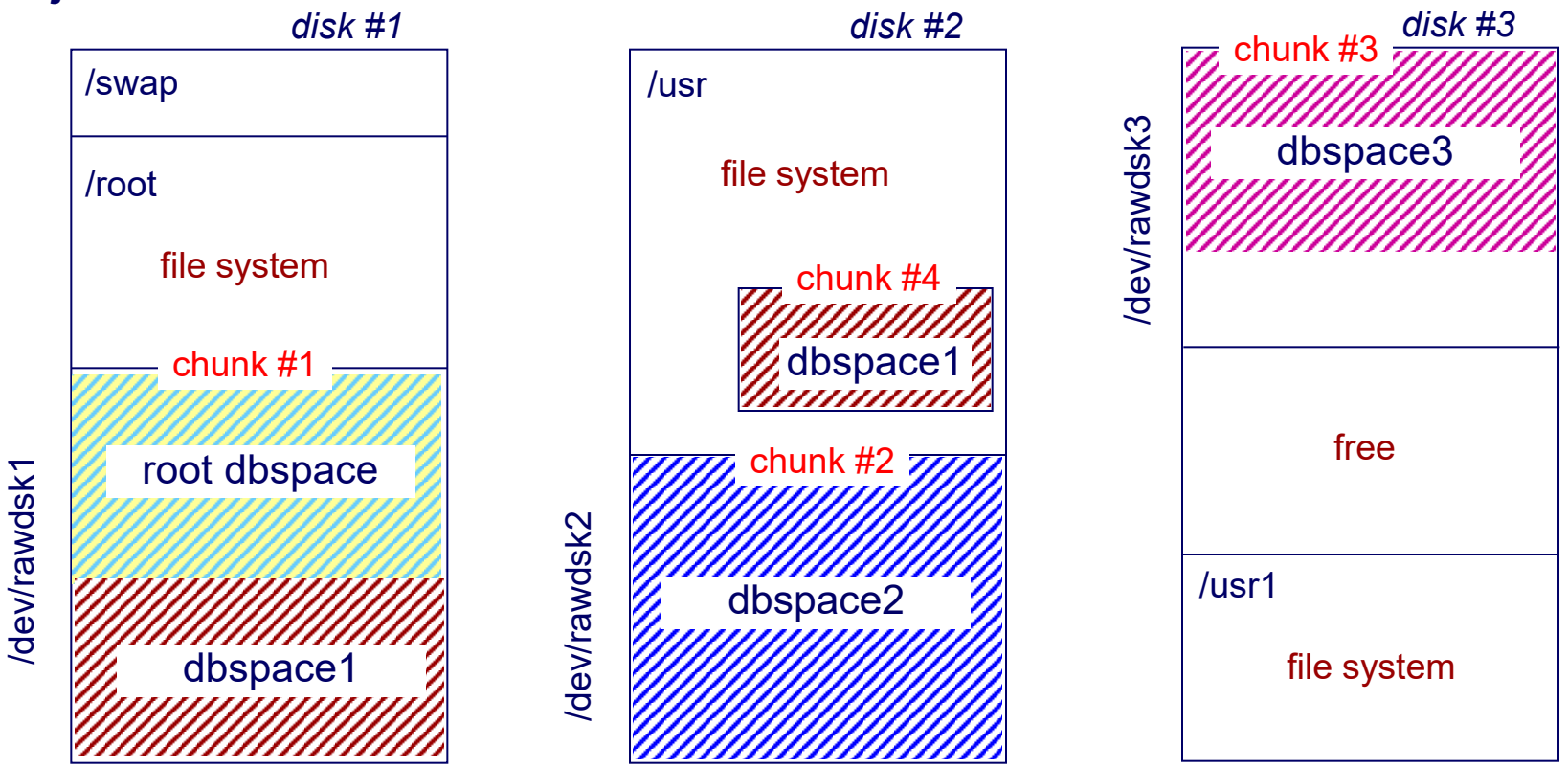
Prostor baze podataka (*database space*)

- naziv, veličinu i smještaj prostora baze podataka određuje administrator SUBP-a
- prostor baze podataka formira se u jednom ili više dijelova jednog ili više grumena. Na taj način se najveća logička jedinica pohrane povezuje s najvećom fizičkom jedinicom pohrane

- administrator baze podataka će kasnije moći odabrati prostor baze podataka u kojem će kreirati objekt baze podataka (npr. relaciju). Na taj način indirektno će odrediti i fizički smještaj tog objekta

Prostor baze podataka

Primjer:



- *root dbspace* je kreiran u dijelu grumena #1
- *dbspace1* je kreiran u dijelu grumena #1 i grumenu #4
- *dbspace2* je kreiran u grumenu #2
- *dbspace3* je kreiran u dijelu grumena #3

Prostor baze podataka

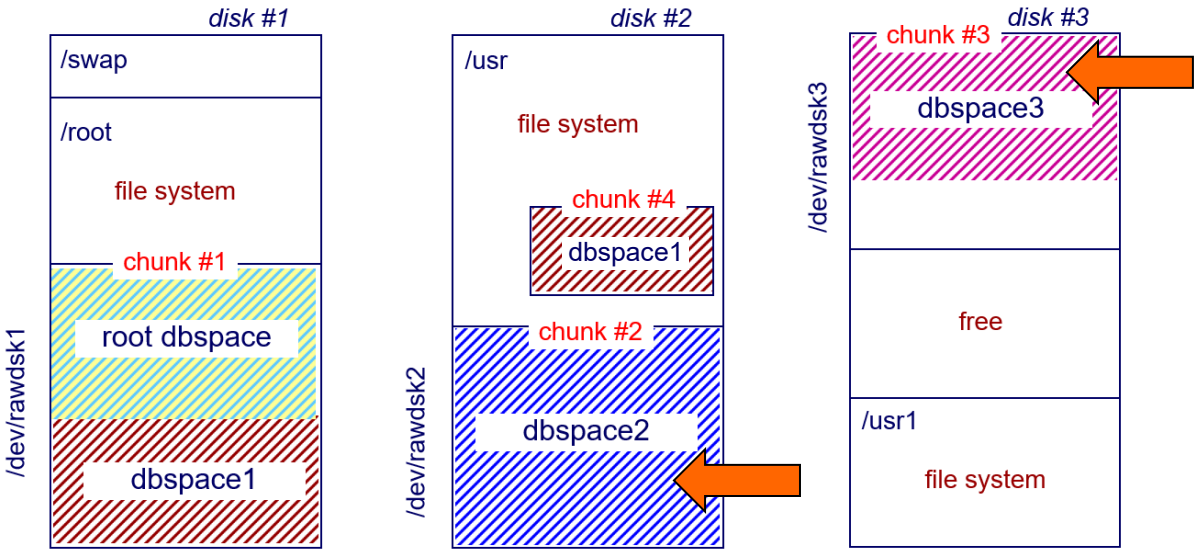
- administrator baze podataka može odrediti gdje će biti smještena baza podataka i pojedine relacije

```
CREATE DATABASE dbName IN dbspace3;
```

  - baza podataka se pohranjuje u prostor *dbspace3*, a također i sve pripadne relacije za koje se CREATE TABLE naredbom ne specificira drugačije

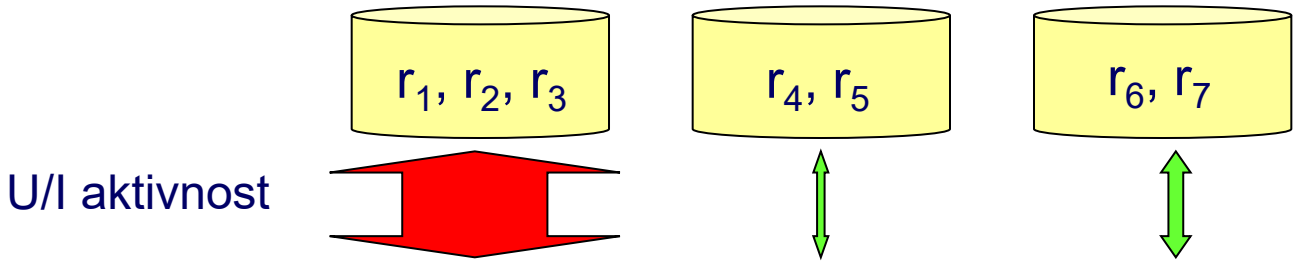
```
CREATE TABLE tabname (...) IN dbspace2;
```

  - relacija se pohranjuje u prostor dbspace2, bez obzira na oblik pripadne CREATE DATABASE naredbe

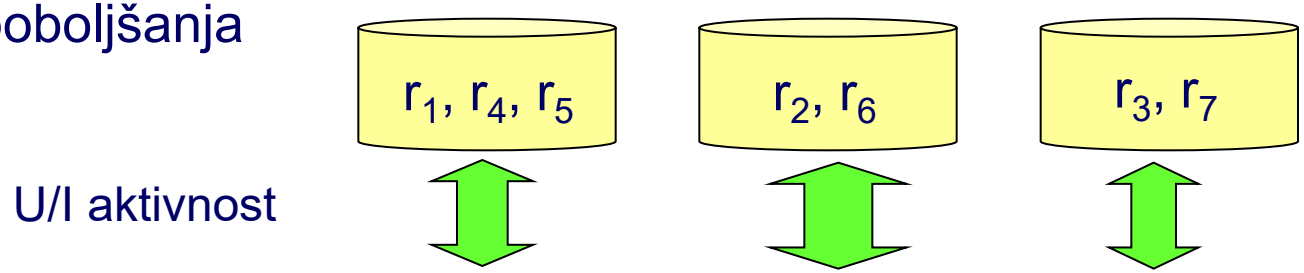


Utjecaj fizičkog smještaja na performanse sustava

- U/I uređaji (*I/O devices*) često predstavljaju uska grla. Npr. neka su
  - relacije r_1, r_2, r_3 relacije s visokom U/I aktivnošću
  - relacije r_4, r_5, r_6, r_7 relacije s niskom U/I aktivnošću
- visoka razina natjecanja (*contention*) za pristup U/I uređajima rezultira pojavom "vrućih točaka", *hotspots*

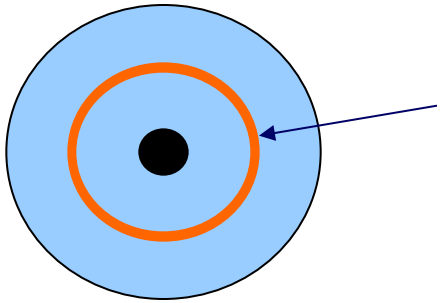


- boljim planiranjem prostora za pohranu, mogu se postići velika poboljšanja



Kako fizičkim smještajem podataka utjecati na performanse?

- nekoliko pravila kojima se treba poslužiti pri planiranju prostora za pohranu. Relacije s najvišom U/I aktivnošću:
  - smjestiti na zasebne U/I jedinice
  - ako postoje U/I jedinice različite brzine, smjestiti ih na najbrže U/I jedinice
  - smjestiti ih čim bliže središnjim cilindrima diska



  - kreirati particiju (npr. *raw device*) u blizini središnjih cilindara diska
  - relaciju kreirati u toj particiji

- kamo smjestiti jednu relaciju koja se među ostalima ističe visokom U/I aktivnošću?
  - fragmentacija (*fragmentation*, *partitioning*) omogućuje smještaj podataka jedne relacije na više U/I uređaja. Koriste se sljedeće sheme fragmentacije:
    - *round-robin*
    - *expression based*

Shema fragmentacije relacije: *round-robin*

- n-torke se smještaju u fragmente na "slučajan" način. Pri tome se fragment određuje kao rezultat hash funkcije nad brojem dobivenim generatorom slučajnih brojeva ili se fragmenti ciklično izmjenjuju (jedna n-torka u prvi fragment, jedna u drugi fragment, ...). Postiže se jednolika raspodjela n-torki.

```
CREATE TABLE ispit (
  jmbag    CHAR(10)
, sifPred  INTEGER
, datIspit DATE
, sifNast  INTEGER
, ocjena   SMALLINT
, PRIMARY KEY (jmbag
               , sifPred
               , datIspit)
    CONSTRAINT pkIspit
) FRAGMENT BY ROUND ROBIN
  IN dbs1, dbs2, dbs3, dbs4;
```

```
SELECT sifPred, AVG(ocjena)
  FROM ispit
  GROUP BY sifPred;
```

- operacije prijenosa blokova (*input*) mogu se obavljati paralelno nad svim U/I uređajima u kojima je pohranjena relacija ispit

- shema je prikladna kada se izvršavaju upiti koji pristupaju velikom dijelu neke relacije (ili, kao u primjeru, svim n-torkama)

Shema fragmentacije relacije: *expression based*

- n-torke se smještaju u fragmente na temelju predikata kojeg zadovoljavaju

```
CREATE TABLE student (
  jmbag     CHAR(10)
, ime       CHAR(20)
, prez      CHAR(20)
, sifFakul  INTEGER
, PRIMARY KEY (jmbag)
     CONSTRAINT pkStudent
) FRAGMENT BY EXPRESSION
  (sifFakul=36)  IN dbs1
, (sifFakul=120)IN dbs2
, (sifFakul=210)IN dbs3
, REMAINDER IN dbs4;
```

```
SELECT * FROM student
  WHERE prez LIKE 'Hor%'
    AND sifFakul=120;
```

- SUBP pristupa samo onim fragmentima koji sadrže podatke koji zadovoljavaju uvjet selekcije (ako uvjet sadrži kriterij iz sheme fragmentacije)
- shema je prikladna kada se često izvršavaju upiti čiji uvjeti za selekciju sadrže kriterij naveden u shemi fragmentacije

Vertikalna fragmentacija relacija

- u relaciji se nalazi 10^6 n-torki

```
CREATE TABLE osoba (
  oib        CHAR(10)
, prez_ime   CHAR(86)
, visina     INTEGER
, zivotopis CHAR(10000)
, PRIMARY KEY (oib)
    CONSTRAINT pkOsoba);
```

- u grubo procijeniti: koliko blokova veličine 2K je potrebno prenijeti (sekundarna→primarna memorija) pri obavljanju sljedećeg upita:

```
SELECT AVG(visina)
  FROM osoba;
```

→ 10^6 * 10100 / 2048 ≈ 4.9 * 10^6 blokova

Vertikalna fragmentacija relacija

- vertikalna fragmentacija
  - stvara se nova relacija s primarnim ključem jednakim kao u originalnoj relaciji, pa se u nju prebacuju atributi koji zauzimaju veliki prostor, a u upitima se ne koriste često

```
CREATE TABLE osoba (
  oib        CHAR(10)
, prez_ime   CHAR(86)
, visina     INTEGER
, PRIMARY KEY (oib)
);
```

```
CREATE TABLE biogr (
  oib        CHAR(10)
, zivotopis CHAR(10000)
, PRIMARY KEY (oib)
, FOREIGN KEY ... osoba...
);
```

- u grubo procijeniti: koliko blokova veličine 2K je potrebno prenijeti (sekundarna→primarna memorija) pri obavljanju sljedećeg upita:

```
SELECT AVG(visina)
  FROM osoba;
```

$\rightarrow 10^6 * 100 / 2048 \approx 4.9 * 10^4$ blokova

- što kada u nekom upitu zatreba i podatak o životopisu?
  - jednostavno obaviti prirodno spajanje relacija osoba i biogr

Literatura:

- H. Garcia-Molina, J. Ullman, J. Widom. Database system implementation. Prentice-Hall International, Inc. Upper Saddle River, New Jersey. 2000.
- A. Silberschatz, H. Korth, S. Sudarshan. Database system concepts, 6th ed. McGraw-Hill. 2010.
- R. Elmasri, S. Navathe. Fundamentals of database systems, 3rd ed. Addison-Wesley Publishing Company. Reading, Massachusetts. 2000.
- IBM Informix Dynamic Server Administrator's Reference, Version 11.1, IBM, 2007.
- IBM Informix Dynamic Server Administrator's Guide, Version 11.1, IBM, 2007.
- Oracle® Database Performance Tuning Guide, 11g Release 2 (11.2)
- Oracle® Database Concepts 11g Release 2 (11.2)